

面向 NPU 核函数的依赖前沿调度与非对称代价溢出规划

补充材料

高成志 黄骏 叶勤

contact@vennai.org hj992881627@outlook.com yq020319@163.com

目录

1 问题定义与评测器语义	2
1.1 输入 DAG 与缓冲区生命周期	2
1.2 P1: 逻辑生命周期峰值	2
1.3 P2: 连续地址与显式溢出工件	2
1.4 静态类别与 P2 目标	2
1.5 P3: 增广 DAG 的流水完成时间	3
1.6 验证、复现与数据范围	3
2 形式化结论及适用边界	3
2.1 标准流量分解与两倍界	3
2.2 固定序驻留间隙模型	4
2.3 从下界到机器可检证书	4
2.4 候选组合的单调性	5
2.5 复杂度与精确后端边界	5
3 求解器实现与完整求解过程	5
3.1 Canonical 可扩展求解器	5
3.2 一个可逐步复算的小图	7
3.3 真实实例 Conv_Case0 的端到端路径	8
3.4 离线代价修复与固定序 oracle	9
3.5 失败、未完成与复现边界	9
4 完整证据与负面结果	9
4.1 公开实例的完整 P1、P2 与 P3 结果	9
4.2 选定消融路径与描述性非加性	11
4.3 $Tr = Vol + Dt$ 下的三类溢出区间	11
4.4 组合内部的 P2–P3 权衡	13
4.5 个案修复与固定序研究 oracle	13
4.6 单实例容量边界	15
4.7 自有合成基准的标准验证复评	16

补充材料概览

本补充材料记录论文“精确模型到启发式”证据链的可复核细节。第 1 节说明 P1/P2/P3 评测语义及其静态有备份/生成分类；第 2 节给出流量恒等式、固定序下界与证书的完整证明及连续地址反例；第 3 节通过教学小图和 Conv_Case0 展示端到端求解过程，并严格区分生产求解器、非统一离线修复个案和固定序研究 oracle；第 4 节报告完整 P1/P2/P3、组件归因、单实例容量扫描、36 例逐实例合成结果、8 例 oracle、工件边界与负面结果。

1 问题定义与评测器语义

本节把输入、输出和评测器之间的契约完整写出。这样做很重要：P1 的“驻留量”、P2 的物理驻留段以及 P3 的流水完成时间是三个不同对象，不能互相替代；同样，评测器中的有备份/无备份分类也不是硬件动态 dirty bit。

1.1 输入 DAG 与缓冲区生命周期

输入为有向无环图 $G = (N, E)$ 。节点集合由三类节点组成：

1. ALLOC 节点创建缓冲区；
2. FREE 节点结束该缓冲区的逻辑生命周期；
3. 其余节点为计算或搬运操作，并列出它所访问的缓冲区、执行流水线与周期数。

每个缓冲区 b 具有唯一标识、大小 s_b 、存储类型 $\tau(b) \in \{L1, UB, L0A, L0B, L0C\}$ 、唯一的分配节点 a_b 和释放节点 f_b 。把操作节点中出现 b 的位置连同 a_b, f_b 称为 b 的强制事件。原始调度 $\pi = (\pi_0, \dots, \pi_{|N|-1})$ 必须恰好包含每个原始节点一次，并满足所有 $(u, v) \in E$ 的 $\text{pos}_\pi(u) < \text{pos}_\pi(v)$ 。

这里有两个容易误读的边界。第一，数据只给出操作访问了哪些缓冲区，并未给出逐次访问的读/写角色。第二，canonical evaluator 把 FREE 也视为强制驻留事件：如果缓冲区此前被换出，就必须先换入，之后才能释放。

1.2 P1：逻辑生命周期峰值

P1 不输出地址或溢出。对每个存储类型，评测器沿原始调度扫描：遇到 ALLOC 就增加相应大小（L0 类型增加一个缓冲区计数），遇到 FREE 就减少它。因而 L1/UB 的指标是 ALLOC/FREE 前缀和的最大值，L0A/L0B/L0C 的指标是同时逻辑活跃的缓冲区最大个数。

该量是逻辑生命周期统计，不是 P2 中某时刻真实占用物理地址的字节数。特别地，一个逻辑活跃缓冲区在 P2 中可以暂时换出；反过来，P1 的峰值也没有包含地址碎片、换出/换入节点或地址复用依赖。因此 P1 只能描述调度造成的逻辑压力，不能单独证明存在合法的连续地址布局。

1.3 P2：连续地址与显式溢出工件

P2 工件由三部分组成：含合成换出/换入节点的增广调度 $\hat{\pi}$ 、每个缓冲区初始驻留段的地址 m_b ，以及按出现顺序记录的 (b, m'_b) 溢出表。第 j 条溢出记录对应两个确定编号的合成节点 o_j (spill-out) 和 i_j (spill-in)； m'_b 是换入后新驻留段的地址。

一个 P2 工件合法，当且仅当它通过以下检查：

1. **覆盖与拓扑**：每个原始节点和每个合成溢出节点恰出现一次，原始 DAG 的全部依赖保持成立；
2. **溢出次序**： $a_b \prec o_j \prec i_j \prec f_b$ ，且 o_j 与 i_j 之间没有访问 b 的操作；同一缓冲区的两次溢出区间不得交叠；
3. **地址范围**：每个驻留段占用半开区间 $[m_b, m_b + s_b)$ 或 $[m'_b, m'_b + s_b)$ ，必须完全落在其存储容量内；
4. **时空不交叠**：同一存储类型中，生命周期在调度位置上重叠的两个驻留段，其地址区间不得相交。

第 4 条同时表达容量和连续地址要求。只验证“某时刻总字节数不超过容量”是不够的：多个小空洞的总和可能足够，却没有一个空洞能容纳新的连续段。

1.4 静态类别与 P2 目标

评测器对缓冲区作静态二分类。若 b 出现在任一 COPY-IN 操作中，则称其为有输入备份（下文简写为 backed/clean）；否则称为生成且无备份（generated/dirty）。这一规则不会随操作执行而改变，也不推断一次访问究竟是读还是写。

有备份缓冲区被中断一次只需从外存重新读入 s_b 字节；无备份缓冲区需先写回再读入，计 $2s_b$ 字节。

若所有溢出记录中两类缓冲区的累计原始体积为 C 与 D ，令 $V = C + D$ ，则额外流量为

$$E = C + 2D = (C + D) + D = V + D. \quad (1)$$

P2 按字典序最小化 $K_2 = (E, n_{\text{spill}}, T)$ ：先比较额外流量，再比较溢出记录数，最后比较完成时间。论文中的 P2 比较均使用与之对应的 P2 官方工件；不能把 P3 官方工件的数值混入该比较。

1.5 P3：增广 DAG 的流水完成时间

P3 使用与 P2 相同的物理合法性条件，但首要目标改为增广调度的完成时间。评测器为每个节点取其全部依赖完成时间、所属硬件流水线可用时间以及所用地址区间可复用时间三者的最大值作为开始时间，再加上该节点周期数。合成 spill-out/spill-in 节点及其依赖也参与模拟；同一流水线串行，复用同一地址区间的新驻留段必须等待旧段结束。最终目标键为 $K_3 = (T, E, n_{\text{spill}})$ 。

因此 P2 与 P3 不是同一目标的两种写法。扩大提前重载窗口可能增加流量，却把搬运隐藏到计算后面并降低 T ；P3 允许这种交换，只要 K_3 更小。本工作只把观测到的候选组合称为流量-时间权衡，不声称得到全局 Pareto 前沿。

1.6 验证、复现与数据范围

本文把三个层次严格分开：模型目标值、生成出的物理工件、canonical evaluator 验证结果。松弛模型的低目标值不是可提交结果；只有地址、溢出依赖和调度均生成成功，且 evaluator 返回零违规，才报告为合法工件。固定序精确后端的 JSON 与对应 schedule/memory/spill 文本共同构成可重放证据；仅含汇总指标的 ledger 需要重跑确定性求解器才能恢复完整工件。

六个公开实例的原始节点数见表 S1。仓库内另有 36 个内部合成实例和 8 个 17 节点固定序 oracle 实例；它们用于边界与机制检查，不能称为额外公开数据集。求解器不按 case 名选择分支；但 portfolio 保留的旧 P1 fallback 含操作类型相关 tie-break，因此不能把整个求解器写成 motif-agnostic（第 3 节给出精确边界）。

表 S1: 六个公开实例的原始 DAG 规模。FA 表示 FlashAttention。

实例	节点数	实例	节点数
Conv_Case0	2,580	Conv_Case1	36,086
FA_Case0	1,716	FA_Case1	6,952
Matmul_Case0	4,160	Matmul_Case1	30,976

2 形式化结论及适用边界

本节给出当前证据能够支持的完整结论。证明刻意区分计账恒等式、容量松弛下界、固定序证书与跨拓扑序全局最优性；后三者不能相互偷换。

2.1 标准流量分解与两倍界

对第 i 次驻留中断，令 v_i 为缓冲区大小；若该缓冲区无输入备份，令 $d_i = v_i$ ，否则令 $d_i = 0$ 。定义 $V = \sum_i v_i$ 、 $D = \sum_i d_i$ 。

命题 1 (计账恒等式与紧界)。 任一合法 P2/P3 工件的额外流量满足

$$E = V + D, \quad V \leq E \leq 2V. \quad (2)$$

在固定 $V > 0$ 时，仅改变被溢出字节的类别构成，两个工件的流量比至多为 2；该上界可达到。

证明。 有备份缓冲区的一次中断收费 v_i , 无备份缓冲区收费 $2v_i = v_i + d_i$ 。逐项求和得 $E = \sum_i (v_i + d_i) = V + D$ 。又因每项 $d_i \in \{0, v_i\}$, 有 $0 \leq D \leq V$, 故 $V \leq E \leq 2V$ 。当所有被溢出字节均有备份时 $D = 0, E = V$; 当它们均无备份时 $D = V, E = 2V$, 两端均可达到。□

命题 1 是评测器的计账性质, 不是因果机制结论。它没有证明生产候选序读取类别标签 (事实上候选序生成不读取), 也没有证明 D 比 V 更重要。任何机理比较都应同时报告 ΔV 与 ΔD ; 超过两倍的收益不可能在固定 V 下仅由类别构成产生。

2.2 固定序驻留间隙模型

固定一个合法原始拓扑序 π 。对每个缓冲区 b , 按 π 中的位置排列其强制事件: $p_{b,0} < p_{b,1} < \dots < p_{b,r_b}$ 。相邻事件之间的开区间定义一个驻留间隙 $g = (b, j)$ 。二元变量 $x_g = 1$ 表示 b 从事件 j 到事件 $j+1$ 连续驻留, $x_g = 0$ 表示允许中断该驻留。令

$$w_g = \begin{cases} s_b, & b \text{ 有输入备份,} \\ 2s_b, & b \text{ 无输入备份.} \end{cases} \quad (3)$$

对存储类型 k 和调度切面 t , 令 $P_{k,t}$ 为该切面上强制必须驻留的字节数, $\mathcal{G}_{k,t}$ 为跨过该切面的同类型可选间隙集合, 容量为 B_k 。固定序加权间隙模型为

$$L^* = \min_x \sum_g w_g (1 - x_g), \quad (4)$$

$$\text{s.t. } P_{k,t} + \sum_{g \in \mathcal{G}_{k,t}} s_g x_g \leq B_k, \quad \forall k, t, \quad (5)$$

$$x_g \in \{0, 1\}.$$

若某个切面已有 $P_{k,t} > B_k$, 则固定序本身不可由溢出修复, 因为同一强制事件所需的 pinned 集已经超过容量。

命题 2 (固定序流量下界)。 式 (4)–(5) 的最优值 L^* 不大于任意具有连续物理地址的合法固定序计划的流量。

证明。 任取一个合法物理计划 A 。若缓冲区在间隙 g 内始终连续驻留, 则令 $x_g = 1$; 否则令 $x_g = 0$ 。在任一切面, A 的强制驻留段与所有被选择的连续驻留间隙确实同时占用该存储类型; 因为 A 合法, 它们的总字节数不超过 B_k , 故构造出的 x 满足所有容量约束。

每个 $x_g = 0$ 的间隙至少包含一次完整驻留中断, 因此 A 在该间隙支付至少 w_g ; 一个人为计划也可能在同一间隙内中断多次, 支付更多。于是 $E(A) \geq \sum_g w_g (1 - x_g) \geq L^*$ 。该论证没有使用 A 的具体地址, 故对所有合法连续地址计划成立。□

2.3 从下界到机器可检验证书

容量松弛丢弃了地址区间的形状信息, 所以即使允许从头全局安排所有地址, 其可行解也未必可打包。下面给出一个离散存储分配反例。容量为 4, 七个段的“活跃时间切片集合/大小”分别为

$$A : \{0\}/2, B : \{0, 1\}/2, C : \{1, 2\}/1, D : \{1, 2, 3\}/1, E : \{2, 3\}/1, F : \{3, 4\}/2, G : \{4\}/2.$$

每个切片的活跃总量依次为 4, 4, 3, 4, 4, 所以全部累计容量约束都成立; 但不存在一组全生命周期固定的连续 offset。证明如下: 切片 0 强迫 A, B 各占一个长度 2 的半区间。不失一般性令 $B = [0, 2)$ 。切片 1 又强迫 C, D 分别占 $[2, 3)$ 与 $[3, 4)$ 。若 $D = [2, 3)$, 则切片 2 的 E 只能落在 $[0, 2)$ 的某个单位位置; 到切片 3, D, E 把剩余空间割裂, 长度 2 的 F 无处放置。若 $D = [3, 4)$, 只有 $E = [0, 1)$ 时切片 3 尚可令 $F = [1, 3)$; 但到切片 4, F 两侧各只剩 1, 长度 2 的 G 无处放置 (其他 E 位置已在切片 3 失败)。左右镜像同理。因此间隙模型只看总体积会接受该组区间, “gap optimal”本身却不是物理可行或 canonical-valid 的证明。

定理 3 (固定序流量证书)。 对给定固定序, 若同时满足:

1. 间隙模型已经证明最优值为 L^* ;
2. 所选间隙被实现为连续地址驻留段, 且全部换出/换入依赖合法;
3. canonical evaluator 返回零违规, 并计算出 $E(A) = L^*$;

则工件 A 在该固定序下具有最小额外流量。

证明。 由命题 2, 任一合法固定序物理计划 A' 均满足 $E(A') \geq L^*$; 由条件 2-3, A 是合法计划且 $E(A) = L^*$ 。故不存在同一固定序下流量更小的合法计划。 $\square$

定理 3 的限定语不可省略。贪心打包或 NoOverlap2D 的“OPTIMAL”只表示其可满足性模型证明找到/确认了布局, 并非又优化了一次流量; 打包失败或超时不推翻 L^* 作为下界。该证书也不优化相同流量下的溢出次数或时间, 所以不能称为完整 P2 最优; 固定序最优更不等于在所有拓扑序上全局最优。

2.4 候选组合的单调性

令 $\mathcal{A}$ 为候选循环中成功生成的工件集合, K 为实现内部按 P2 或 P3 语义计算的字典序键。生产求解器返回 $A^* = \arg \min_{A \in \mathcal{A}} K(A)$ 。

命题 4 (候选扩充的内部键单调性)。 若新增成功生成的候选集合 $\mathcal{B}$, 则

$$\min_{A \in \mathcal{A} \cup \mathcal{B}} K(A) \leq_{\text{lex}} \min_{A \in \mathcal{A}} K(A). \quad (6)$$

证明。 $\mathcal{A} \subseteq \mathcal{A} \cup \mathcal{B}$, 右侧原有最优候选仍在新集合中; 在更大集合取最小值不可能更差。 $\square$

这是 portfolio 对内部选择键的单调性, 而不是新候选有效性的因果证据或合法性定理。要把返回值称为合法工件安全性, 还需候选生成器本身正确, 或对最终返回工件运行 canonical evaluator; 生产循环没有逐候选调用后者, 本文实验只报告最终验证为零违规的工件。特别地, 若组合包含一个强基线序, 最终结果不劣于该序只是集合包含关系; 仍需受控消融才能把改进归因到某个排序、地址或 victim 组件。生产选择直接计算真实 K_2/K_3 , 不依赖 overflow-area 代理目标定理。

2.5 复杂度与精确后端边界

对固定候选数、victim 策略数和提前重载窗口数, 可扩展路径依次执行拓扑排序、驻留扫描、地址放置及时间模拟。当前放置器在一次请求上可能扫描驻留集合或空闲区间, 因此以缓冲区数计可保守写成二次项; portfolio 只乘一个固定常数。该陈述是实现上界, 不是线性时间保证。

加权间隙选择和后备 NoOverlap2D 在最坏情况下均为指数复杂度。因此 CP-SAT 后端目前是离线研究 oracle, 不属于 canonical solve。将来若集成, 必须设置规模阈值或超时, 并在求解未完成、打包失败或验证失败时回退到已经验证的可扩展工件。现有实验中的非零 gap、timeout 和未运行项必须原样报告, 不能用成功个案外推为普遍精确性。

3 求解器实现与完整求解过程

本节先区分三种角色, 再把“DAG 如何变成可验证工件”逐步展开。三种角色是: 默认使用的可扩展组合、只用于机制研究的离线代价修复, 以及固定拓扑序的 CP-SAT oracle。后两者都不是生产 solve 的隐藏阶段。

3.1 Canonical 可扩展求解器

生产实现为 P2/P3 生成四个结构拓扑序: unlock-frontier、capfit-id、Pl memory order 和 id-raw。只有第一个是 v2 新增排序; 其余三个提供历史与结构多样性, 不能重新命名为代价感知排序。四个排序均不读取 backed/generated 标签。类别信息只进入两类 victim 评分以及完整候选的真实流量计算与最终选

择。这里的“纯结构”限于新增的 `unlock-frontier`: `portfolio` 保留的旧 P1 fallback 对特定输入形态含 D2S、COPY-IN 和 MOVE 相关 tie-break。因此本文不把整个 canonical solver 称为 motif-agnostic；能成立的窄结论只是 `frontier` 不读取实例名、操作模式或缓冲区类别。

`unlock-frontier` 先调度就绪 FREE，再调度最小编号的就绪操作；仅剩 ALLOC 时，寻找编号最小且“全部尚未完成前驱都已经就绪”的消费者，并完成其前驱组。这样可避免连续的一输入搬运长期饿死一个多输入操作。

Algorithm 1 依赖前沿拓扑序

Require: DAG 实例 I

```

1: 将零入度节点划分为  $Q_f, Q_o, Q_a$ ;  $S \leftarrow []$ 
2: while  $Q_f \cup Q_o \cup Q_a \neq \emptyset$  do
3:   if  $Q_f \neq \emptyset$  then
4:      $v \leftarrow Q_f$  中最大编号节点
5:   else if  $Q_o \neq \emptyset$  then
6:      $v \leftarrow Q_o$  中最小编号节点
7:   else
8:      $u \leftarrow$  尚未完成前驱全在  $Q_a$  中的最小编号消费者
9:      $v \leftarrow u$  的最小编号就绪前驱
10:    if 不存在这样的  $u$  then
11:       $v \leftarrow Q_a$  中最小编号节点
12:    end if
13:  end if
14:  将  $v$  追加至  $S$ ; 更新入度及  $Q_f, Q_o, Q_a$ 
15: end while
16: return  $S$ 

```

每个拓扑序分别运行真正的 best-fit 地址放置：从所有足够大的空闲区间中选择长度最小者，长度相同时选择地址最小者；释放后合并相邻空闲区间。若没有连续空洞，放置器从当前不被下一操作 pinned 的缓冲区中选择 victim，插入 spill-out，释放其整段地址，直至请求可放置。需要再次访问或 FREE 时插入 spill-in，并允许换入到新地址。

生产组合比较 `dist-size-cost` 与 `share-adaptive-25` 两种 victim 策略。前者联合下次使用距离、大小和静态流量代价；后者在有备份字节稀少或纯生成缓冲区尺寸高度异质时切换优先级。P2 枚举提前重载窗口 $H \in \{0, 5, 40\}$ ，P3 枚举 $H \in \{0, 5, 40, 80, 120\}$ 。提前重载只有在已有足够连续空洞时才发生，它本身不驱逐其他驻留缓冲区；但它会改变之后的驻留状态，因而仍须把每个窗口作为完整候选重新评价。

Algorithm 2 P2/P3 有界工件组合**Require:** 实例 I , 问题编号 $q \in \{2, 3\}$

- 1: $\mathcal{O} \leftarrow \{\text{frontier, capfit-id, P1, id-row}\}$
- 2: $\mathcal{P} \leftarrow \{\text{dist-size-cost, share-adaptive-25}\}$
- 3: $\mathcal{H} \leftarrow \{0, 5, 40\}$; 若 $q = 3$, 加入 80, 120
- 4: $A^* \leftarrow \emptyset$
- 5: **for** $(O, P, H) \in \mathcal{O} \times \mathcal{P} \times \mathcal{H}$ **do**
- 6: 生成拓扑序 π_O
- 7: 用 best-fit、策略 P 和窗口 H 生成地址与溢出工件 A
- 8: **if** 放置失败 **then**
- 9: 跳过该不可行组合成员
- 10: **end if**
- 11: 计算 E, n_{spill}, T (候选循环内部不调用 canonical evaluator)
- 12: 按 $K_2 = (E, n_{\text{spill}}, T)$ 或 $K_3 = (T, E, n_{\text{spill}})$ 更新 A^*
- 13: **end for**
- 14: 返回 A^* ; 实验 harness 对最终工件运行 canonical evaluator

算法 2 最多考察 $4 \times 2 \times |\mathcal{H}|$ 个组合成员; 放置失败者被跳过。它对成功候选生成地址/溢出工件并计算真实目标, 不用 live-area 或 overflow-area 代理量作最终选优。生产 solve 的候选循环本身不逐个调用 canonical evaluator; 实验 harness 对最终选中工件作标准验证, 本文只报告零违规结果。表 S2 给出正文伪代码省略但会影响复现的确定性规则。

表 S2: Canonical 求解器的固定常数与边界规则。

机制	数值	精确作用
前沿平局裁决	FREE 最大 ID; ALLOC 键为 (最小可解锁消费者 ID, ALLOC ID); 无可操作最小 ID	解锁消费者时第一项取无穷。
下次使用截断	2^{20}	形成任一 victim 评分前, 截断按基础序位置计算的距离。
尺寸异质性	最大/最小 ≥ 8	若该存储类型完全没有 backed 缓冲, 自适应策略优先释放大连续段; 否则保持距离优先。
backed 占比	$\leq 25\%$	有可驱逐 backed victim 且其字节占可驱逐驻留字节不超过阈值时, 优先使用这一低价储备。
提前重载	距离 $\leq H$	仅使用已经存在的合适空洞; 预取不主动制造 victim。
pinned 搬移	每请求至多 100 次	只有碎片化且无非 pinned victim 时, 溢出并立即换入最小 pinned 缓冲; 超过上限则该候选失败。

3.2 一个可逐步复算的小图

表 S3 给出一个按当前 best-fit/spill 实现与 canonical evaluator 规则手工展开的教学例子, 用来明确溢出节点、地址与计费如何共同形成 P2 工件; 它不是运行日志、公开基准或 oracle 数据, 也不用于论文数值主张。把七个原始节点按下列顺序编号, 并以相邻节点之间的六条边 $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$ 构成一条 DAG 链; 因此原始拓扑序可直接复核且唯一。设某存储容量为 4, 缓冲区 a 为 2 字节 backed 输入, b 为 3 字节 generated 中间量。原始合法序为

ALLOC(a), COPYIN(a), ALLOC(b), PRODUCE(b), FREE(b), USE(a), FREE(a).

a 在 COPY-IN 后到 USE 前没有强制访问。分配 b 时总逻辑活跃量为 5, best-fit 无法找到 3 字节空洞; 选择换出 a 后, b 可占 $[0, 3)$ 。由于 FREE 是强制事件, b 必须保持驻留到其 FREE; 随后再把 a 换入并执行 USE。这里选择 backed 的 a , 一次中断只收费 2 字节。

表 S3: 容量为 4 的教学小图: 从原始序到增广物理工件。 o_a, i_a 为合成换出/换入节点。

步	执行事件	事件后驻留段	空闲区间	累计 E
1	ALLOC(a)	$a : [0, 2)$	$[2, 4)$	0
2	COPYIN(a)	$a : [0, 2)$	$[2, 4)$	0
3	o_a	无	$[0, 4)$	2
4	ALLOC(b)	$b : [0, 3)$	$[3, 4)$	2
5	PRODUCE(b)	$b : [0, 3)$	$[3, 4)$	2
6	FREE(b)	无	$[0, 4)$	2
7	i_a	$a : [0, 2)$	$[2, 4)$	2
8	USE(a)	$a : [0, 2)$	$[2, 4)$	2
9	FREE(a)	无	$[0, 4)$	2

该工件的溢出表只有 $(a, 0)$ 一项, 所以 $n_{\text{spill}} = 1, C = 2, D = 0, V = 2, E = 2$ 。验证器还会检查原始依赖、 $\text{ALLOC}(a) \prec o_a \prec i_a \prec \text{FREE}(a)$ 、 o_a 与 i_a 之间无 a 的使用, 以及所有半开地址区间均不越界或时空重叠。若换出的是同尺寸 generated 缓冲, 流量会是 4; 这正是命题 1 的单位代价差异, 而非“排序自动使用类别”的证据。

3.3 真实实例 Conv_Case0 的端到端路径

真实实例不能逐节点印出 2,580 行, 表 S4 因而按求解角色展示从候选序到验证工件的完整方法路径。可扩展 v2 最多考察 24 个 P2 组合成员, 逐一执行“排序–best-fit–victim–溢出插入–内部计价/计时”, 再由实验 harness 对最终选中工件作 canonical 验证。该工件零违规, $E = 66,828$ 、140 次溢出; 其流量可直接分解为 $C = 12,672, D = 27,078, V = 39,750$, 故 $E = V + D = 39,750 + 27,078 = 66,828$ 。

表 S4: Conv_Case0 的方法路径。exact 行只证固定 unlock-frontier 序的流量最优。

角色	工件/搜索范围	E	溢出数	可作出的结论
历史前身	iter038	88,044	339	可扩展 P2 前身
生产	scalable v2, 24 候选	66,828	140	canonical-valid 生产结果
离线修复	10,000 次随机单节点 proposal	65,532	137	Conv0 个案, 非默认阶段
固定序 exact	frontier gap+packing	57,408	112	下界相等、打包成功、固定序流量证书
官方	官方 P2 工件	73,500	141	同口径比较对象

固定序 exact 行进一步展示证书链: 间隙 CP-SAT 在 0.88 秒的记录运行中给出下界与目标均为 57,408; 连续地址打包状态为 OPTIMAL (这里只表示可满足性), 标准 schedule/memory/spill 文件生成成功, canonical evaluator 返回零违规且同样计算出 57,408。由定理 3, 它证明该 unlock-frontier 序的最小流量; 它没有证明 112 是同流量下最少溢出数, 也没有搜索所有拓扑序。作为排序敏感性的受控证据, 同一 planner 固定为 P1 序时证明的流量最优值为 81,504; 这一对比只对 Conv0 成立。

3.4 离线代价修复与固定序 oracle

离线修复脚本观察高代价溢出前沿，提出保持依赖合法的节点或依赖链移动，重新运行物理规划，并只接受 P2 键严格改善的 proposal。实验预算并不统一：Conv_Case0 使用 10,000 次随机单节点 hill search；Conv_Case1 使用 beam 宽度 10、两轮 frontier search；两个 FA 实例各尝试 2,000 个随机 proposal 但没有接受移动；两个 Matmul 实例没有运行修复。因而它只能说明某些个案中代价语义可能影响排序，不能称为六例统一算法或普遍改进。

固定序 oracle 先解式 (4) 的加权间隙选择，再进行若干确定性贪心打包；必要时调用独立 NoOverlap2D 可满足性模型。最后它输出与生产求解器相同格式的 schedule/memory/spill 工件，并调用同一 evaluator。8 个内部 17 节点 oracle 实例上，生产求解器在其自身所选固定序的流量依次为 352、1,408、704、704、352、1,408、704 和 704；每一项都与该固定序的 CP-SAT 下界相等且通过验证。这是 8 个离散小图的固定序吻合，不是大规模扩展性或跨序全局最优性证据。

3.5 失败、未完成与复现边界

完整求解过程也包括失败状态，而不是只保留成功证书：

- FA_Case1 的固定序工件为 canonical-valid FEASIBLE: $E = 32,512$ ，下界 31,936，gap 为 576；不能称为 certified optimal；
- Matmul_Case0 的可复现 120.09 秒运行给出 FEASIBLE 34,816、下界 29,952；同一序另有合法启发式工件 34,688，故审计区间为 [29,952, 34,688]，旧的“34,688 OPTIMAL”说法已撤销；
- Conv_Case1 未在研究预算内完成 exact+packing，Matmul_Case1 未运行；
- P3 在六个公开实例中对官方时间是 5 胜 1 负，Conv_Case1 为 1,118,687 对 1,073,322，不能写成普遍占优；
- 汇总 JSON 可核对指标和验证状态，但若其旁边没有配套文本工件，独立 replay 需要重跑确定性求解器。固定序证书则以机器 JSON 和匹配的 schedule/memory/spill 文件为准，旧的手写状态不得覆盖机器记录。

最后，生产路径的地址放置和驻留扫描在固定 portfolio 下是可扩展实现；CP-SAT 的间隙选择与后备 packing 最坏情况下为指数复杂度。当前 canonical solve 没有 CP-SAT 规模阈值、超时切换或回退集成，故不能把 oracle 的成功结果描述为默认求解器能力。

4 完整证据与负面结果

本节展开主文中压缩呈现的实验，并为每张表明目标和工件角色。P2 与 P3 来自使用不同字典序键的独立搜索；精确结果仅是研究 oracle 的证据，不属于生产输出。

4.1 公开实例的完整 P1、P2 与 P3 结果

六个公开实例均来自 2025 年“华为杯”第二十二届中国研究生数学建模竞赛 A 题发布的 DAG；本节后续报告的合成族是另行构造的内部测试集。

表 S5 补齐当前求解器的 P1 逻辑生命周期指标。该表由 results/autoresearch_v2/round12_audited_p1.json 重新生成，六行均经 evaluator 检查为零违规；括号内为相匹配的官方 P1 工件。这里的 L1/UB 是 ALLOC-FREE 前缀和峰值，L0A/B/C 是同时逻辑活跃的缓冲区个数，均不是 P2 溢出后的物理驻留轨迹。当前序在两个 Conv 实例上略降 L1 峰值，却显著增加 L0B 活跃计数；这一负面结果说明 P1 各分量及其时间不能被压缩成“全面优于官方”的单一结论，也不能用于替代后续 P2/P3 工件比较。

P2 五胜一平，中位降幅 0.866%、最大降幅 9.08%；Matmul_Case1 的流量持平，但以时间次级键获胜。P3 五胜一负，中位降幅 3.77%；Conv_Case1 回退 4.23%。这些统计只比较生产求解器与目标相匹配的官方 P2 或 P3 工件，不混入修复个案或固定序 oracle。表中 JSON 是指标与验证 ledger，并未在同目录持久

表 S5: 六个公开实例的 P1 逻辑指标，格式为本文（官方）。时间为评测器计算的调度周期。

实例	L1	UB	L0A	L0B	L0C	时间
Conv_Case0	6,912 (7,488)	0 (0)	0 (0)	300 (1)	0 (0)	394,430 (359,570)
Conv_Case1	13,786 (14,040)	0 (0)	0 (0)	5,264 (1)	0 (0)	650,044 (573,395)
FA_Case0	256 (3,328)	8,258 (1,428)	24 (1)	32 (1)	4 (1)	31,063 (31,429)
FA_Case1	256 (5,376)	33,010 (2,468)	112 (1)	128 (1)	8 (1)	112,458 (113,445)
Matmul_Case0	128 (9,216)	0 (0)	57 (1)	448 (1)	8 (1)	82,308 (82,773)
Matmul_Case1	128 (34,816)	0 (0)	241 (1)	3,840 (1)	16 (1)	629,076 (583,525)

表 S6: 生产求解器的完整 P2 记录。Vol 为原始溢出体积，Dt 为生成缓冲体积；wall 为一次审计运行的求解器墙钟记录，不是重复测量的性能估计。六行均通过 canonical evaluator，零违规。

实例	Tr	次数	Vol	Dt	T	调度长度	官方 Tr	结果	wall (s)
Conv_Case0	66,828	140	39,750	27,078	521,034	2,860	73,500	胜	0.552
Conv_Case1	72,734	1,987	69,731	3,003	1,154,672	40,060	73,240	胜	43.055
FA_Case0	3,584	14	1,792	1,792	44,956	1,744	3,692	胜	0.263
FA_Case1	32,512	127	20,480	12,032	164,166	7,206	32,840	胜	2.216
Matmul_Case0	34,688	271	34,688	0	192,380	4,702	34,944	胜	0.927
Matmul_Case1	460,800	3,600	460,800	0	1,783,083	38,176	460,800	平	20.973

表 S7: 生产求解器的完整 P3 记录。P3 以时间优先；官方流量与时间均来自相匹配的 P3 工件。wall 同样只是一轮审计运行记录。六行均通过 canonical evaluator，零违规。

实例	Tr	次数	T	调度长度	官方 Tr	官方 T	时间结果	wall (s)
Conv_Case0	69,712	147	515,634	2,874	73,500	535,312	胜	1.106
Conv_Case1	72,742	2,015	1,118,687	40,116	86,690	1,073,322	负	61.133
FA_Case0	4,096	16	36,344	1,748	4,128	46,761	胜	0.434
FA_Case1	34,304	134	152,364	7,220	32,840	193,059	胜	3.119
Matmul_Case0	34,944	273	186,820	4,706	34,944	194,331	胜	1.585
Matmul_Case1	460,928	3,601	1,771,383	38,178	460,800	1,800,218	胜	36.963

化六例生产运行的完整 schedule/memory/spill 三件套；完整工件的独立 replay 需要重跑确定性求解器。因此 wall 只用于记录本次运行，不能据此声称稳定吞吐或扩展律。

4.2 选定消融路径与描述性非加性

表 S8 给出固定 $H = 0$ 的选定受控配置。这里的参考值不是实际晋级版本，“选定”行也不是完整 canonical 组合；round7 的每个配置只按 (E, n) 选择。该设计并非完整 factorial，缺少两个成对组合，因此只能观察一条具有非加性的配置路径，不能识别孤立主效应或交互系数。

表 S8: 选定的固定 $H = 0$ 流量消融配置。BF 为 best-fit，F 为 frontier；“选定”还启用自适应策略，但不是 factorial 估计。

实例	实际前身	$H = 0$ 参考	仅 BF	仅 F	选定	生产差值
Conv_Case0	88,044	88,044	87,660	85,696	66,828	-21,216
Conv_Case1	72,520	73,348	72,734	73,348	72,734	+214
FA_Case0	3,904	3,904	3,620	3,904	3,584	-320
FA_Case1	32,920	32,920	32,852	32,920	32,512	-408
Matmul_Case0	34,688	34,688	34,688	34,688	34,688	0
Matmul_Case1	460,800	460,800	460,800	460,800	460,800	0

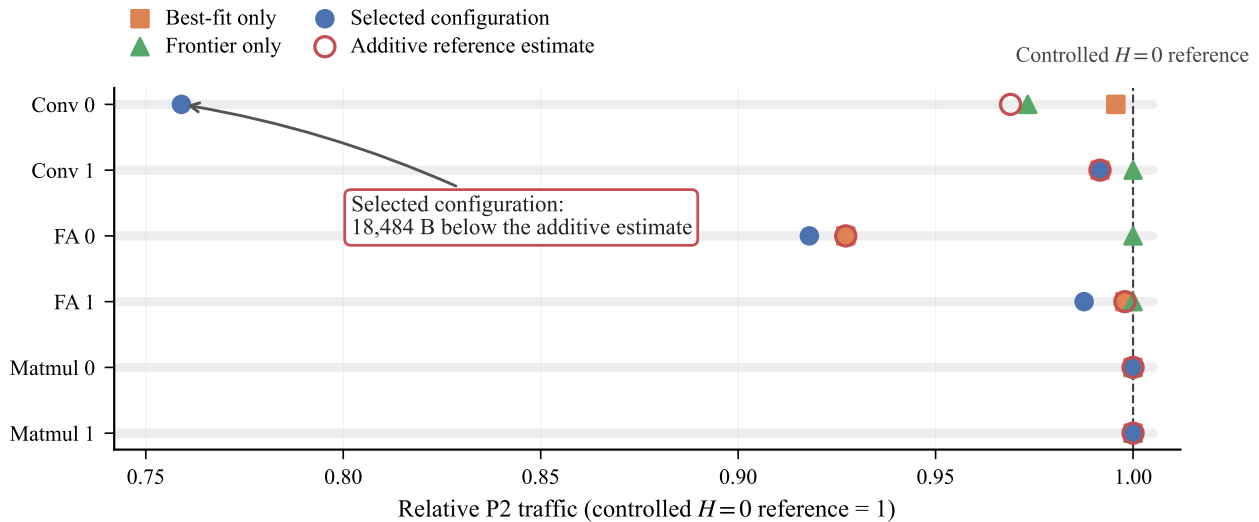


图 S1: 选定的受控 $H = 0$ 配置相对参考值的 P2 流量。空心圆表示把“仅 BF”和“仅 F”的实测变化直接相加所得的参照预测；它只用于显示这条配置路径的非加性，不作 factorial 归因。相对实际前身，选定配置为三胜、两平、一负。

Conv_Case0 中，仅根据 best-fit-only 与 frontier-only 的测得变化作朴素加法会预测 85,312 字节，而启用全部机制的选定 $H = 0$ 配置达到 66,828，描述性偏差为 18,484 字节。这说明测得配置路径非加性，但不能称为已隔离的“协同”：选定配置还含自适应策略，且缺失的成对单元阻止 factorial 归因。FA_Case0 也从 first-fit 下 frontier+adaptive 的 4,608 变为加入 best-fit 后的 3,584，表明策略结果依赖地址放置。Conv_Case1 则是另一边：相对受控 $H = 0$ 参考有所改善，却比实际前身差 214 字节。两个 Matmul 行不变。证据只支持完整配置评价，不支持普适的单模块收益。

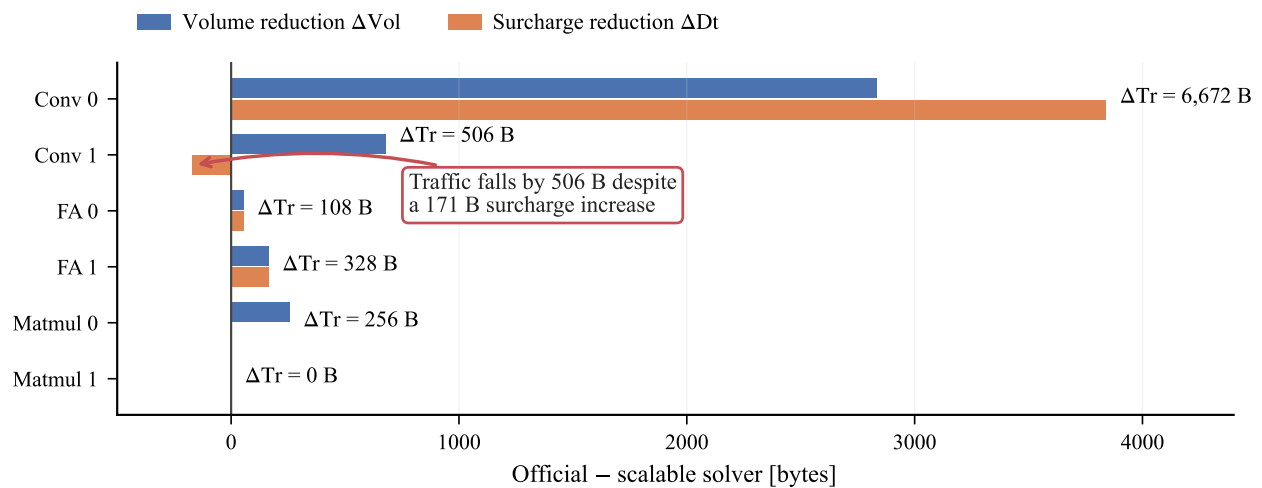
4.3 $Tr = Vol + Dt$ 下的三类溢出区间

表 S9 将每个可扩展 P2 输出与对应官方 P2 工件配对。表中 Vol 为原始总溢出体积，Dt 为其中生成/无备份部分，故每行满足 $Tr = Vol + Dt$ 。差值定义为官方减本文，因此 $\Delta Tr = \Delta Vol + \Delta Dt$ 。这种配对记账不预设体积或附加代价哪一个是唯一机制。

这些行仍覆盖三类区间：FA_Case0 为全生成 ($Dt = Vol$)，两个 Matmul 为全有备份 ($Dt = 0$)，Conv 与 FA_Case1 为混合。五个严格 P2 胜例全部降低 Vol。Dt 在 Conv_Case0、FA_Case0 和 FA_Case1 中下降，

表 S9: 全部公开实例的配对 P2 字节分解。差值为官方减本文，并满足 $\Delta Tr = \Delta Vol + \Delta Dt$ 。

实例	官 Vol	官 Dt	官 Tr	本文 Vol	本文 Dt	本文 Tr	ΔVol	ΔDt	ΔTr
Conv_Case0	42,582	30,918	73,500	39,750	27,078	66,828	2,832	3,840	6,672
Conv_Case1	70,408	2,832	73,240	69,731	3,003	72,734	677	-171	506
FA_Case0	1,846	1,846	3,692	1,792	1,792	3,584	54	54	108
FA_Case1	20,644	12,196	32,840	20,480	12,032	32,512	164	164	328
Matmul_Case0	34,944	0	34,944	34,688	0	34,688	256	0	256
Matmul_Case1	460,800	0	460,800	460,800	0	460,800	0	0	0

**图 S2:** 各实例相对官方工件的 P2 分项记账。五个严格胜例都降低总溢出量 Vol，生成附加量 Dt 则有升有降；两项带符号变化之和就是净流量降幅，即 $\Delta Tr = \Delta Vol + \Delta Dt$ 。

在 Matmul_Case0 中保持为零, 却在 Conv_Case1 中上升 171 字节; Conv_Case1 仍获胜, 是因为 677 字节的体积下降抵消了该回退, 即 $506 = 677 - 171$ 。FA_Case0 的 Vol, Dt 同降 54, 类别占比未变; Matmul_Case0 在 $Dt = 0$ 时完全靠 Vol 降 256 获胜。这些配对事实直接排除了统一的构成解释。

4.4 组合内部的 P2-P3 权衡

P2 按 (Tr, n, T) 选择, P3 按 (T, Tr, n) 选择。表 S10 在同一实例、同一有界组合内比较两个目标分别选出的本文工件; 这控制了求解器和实例, 但不建立全局 Pareto 前沿。

表 S10: 组合内部的目标权衡。Tr₂, T₂ 属于 P2 选中工件, Tr₃, T₃ 属于独立选出的 P3 工件。

实例	Tr ₂	Tr ₃	Tr ₃ - Tr ₂	T ₂	T ₃	T ₃ - T ₂
Conv_Case0	66,828	69,712	2,884	521,034	515,634	-5,400
Conv_Case1	72,734	72,742	8	1,154,672	1,118,687	-35,985
FA_Case0	3,584	4,096	512	44,956	36,344	-8,612
FA_Case1	32,512	34,304	1,792	164,166	152,364	-11,802
Matmul_Case0	34,688	34,944	256	192,380	186,820	-5,560
Matmul_Case1	460,800	460,928	128	1,783,083	1,771,383	-11,700

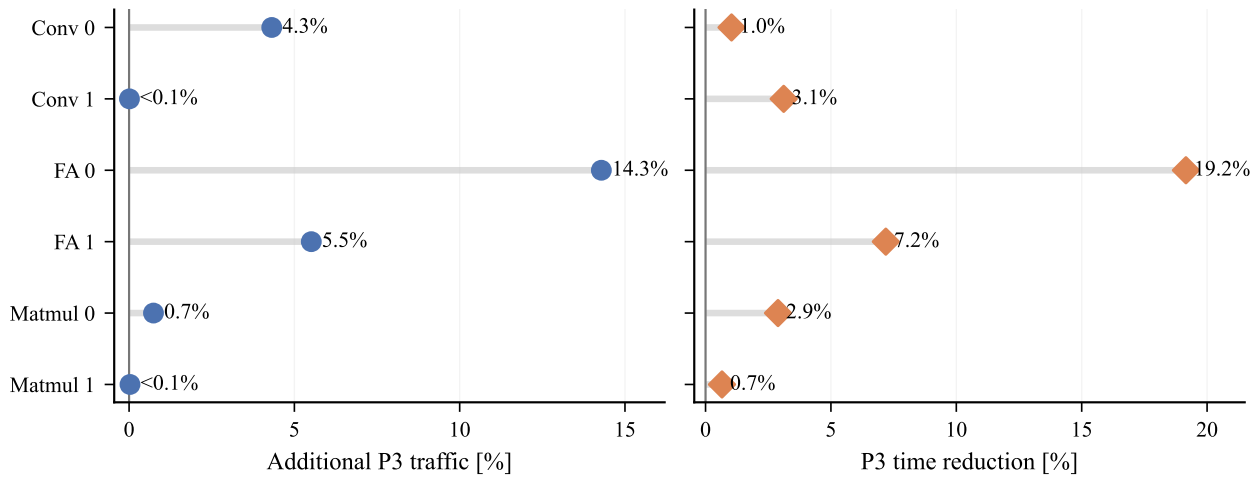


图 S3: 同一组合内, 从 P2 选中工件切换到独立选出的 P3 工件所发生的变化。左右两栏分别给出新增流量与时间降幅; 六例均以更多流量换取更短时间。这只是有界组合内的观察, 不是全局 Pareto 主张。

P3 搜索在全部公开实例上都比 P2 输出接受更多流量, 范围从 Conv_Case1 的 8 字节到 Conv_Case0 的 2,884 字节, 同时每个实例上都比本文 P2 工件更快。Conv_Case1 是接近零流量变化的鲜明例子: 多 8 字节对应少 35,985 周期。这才是目标解耦的直接组合内部证据。官方 P2 与官方 P3 是不同基线, 因此“Conv_Case1 相对官方 P2 获胜、相对官方 P3 失败”只能提醒我们不能跨目标转移胜负结论, 不能用来推导上述权衡。

4.5 个案修复与固定序研究 oracle

表 S11: 两个非统一的代价感知拓扑序修复个案。搜索过程与预算不同, 不构成一个覆盖六例的统一方法阶段。

实例	基础 Tr	修复 Tr	ΔTr	搜索预算
Conv_Case0	66,828	65,532	-1,296	10,000 次单节点 hill, seed 0
Conv_Case1	72,734	70,940	-1,794	spill-frontier beam 10, 两轮

Conv_Case0 与 Conv_Case1 的收益因此来自两个不同修复原型, 并非在统一预算下评价的同一算法。

FA_Case0 和 FA_Case1 各自另做 2,000-proposal、seed-0 的 hill 小预算探针，均未接受移动；该负面探针不能与两个 Conv 预算直接比较。Matmul 没有运行修复搜索，其他位置出现的 Matmul 数字只是复用结构序。

表 S12: 固定序研究 oracle 的流量证据。“证书”要求机器可读 gap 结果、打包后的 schedule/memory/spill 工件、canonical 验证及流量等于下界。

实例	固定序	流量	下界	差距	证据状态
Conv_Case0	unlock-frontier	57,408	57,408	0	流量证书
Conv_Case0	旧 P1	81,504	81,504	0	流量证书
FA_Case0	id-raw	3,584	3,584	0	流量证书
FA_Case1	capfit-id	32,512	31,936	576	机器可检可行解
Matmul_Case0	capfit-id	34,816	29,952	4,864	机器可检可行解
Conv_Case1	–	–	–	–	run-log 打包超时
Matmul_Case1	–	–	–	–	未运行

Conv_Case0/unlock-frontier 的证书含 112 次溢出，耗时 0.88 秒；同一 planner 还以 81,504、328 次溢出、19.49 秒证明旧 P1 序的固定序流量最优。二者在同一实例、同一 planner 下相差 24,096 字节，严格暴露固定序瓶颈，但不声称跨拓扑序全局最优。FA_Case0 是第三份机器可检流量证书，并把证据覆盖扩展到第二个公开实例（14 次溢出，2.17 秒）。FA_Case1 与 Matmul_Case0 均有通过 canonical 验证的打包工件，但正的下界差距使其只能称可行/暂定，旧 Matmul_Case0 34,688 最优结论撤销。CP-SAT 仍是研究 oracle，canonical solve 不调用它；这些证书也不优化同流量下的次数或时间。

两条驻留轨迹及其证书值直接支撑序空间的机制结论，因此移至正文展示。

4.6 单实例容量边界

表 S13: Conv_Case0 的 L1 容量扫描。不可行行在强制驻留事件处已超过容量，并非算法失败。

L1 容量	选定 Tr	无感拓扑序 Tr	比值	选中拓扑序
2,048	–	–	–	强制驻留不可行
2,560	–	–	–	强制驻留不可行
3,072	200,960	272,010	1.354	capfit-id
3,584	101,924	244,868	2.402	unlock-frontier
4,096	66,828	210,876	3.156	unlock-frontier
6,144	20,744	109,116	5.260	unlock-frontier
8,192	576	71,144	123.514	unlock-frontier
12,288	0	13,830	–	unlock-frontier
16,384	0	0	–	unlock-frontier

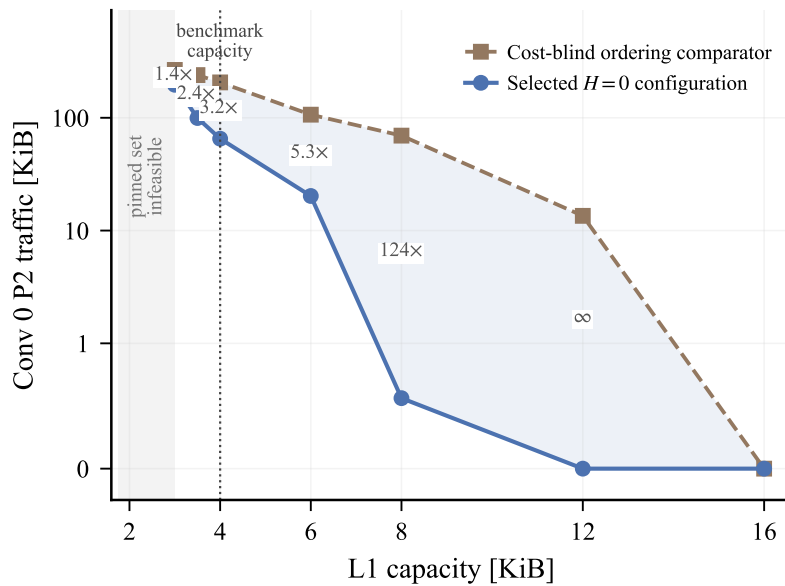


图 S4: Conv_Case0 的受控 $H = 0$ L1 容量扫描（纵轴 symlog）。曲线间标注为比较器与选定配置的流量比值；阴影带仅表示该受控切片中与拓扑序选择相关的差距。容量低于 3,072 字节时强制驻留集本身超容。

在该扫描的所有可行容量上，受控固定- $H = 0$ 选择从未输给代价无感拓扑序比较器。选定侧比较四种结构序、两种 victim 策略与 best-fit；比较器固定 cp-free-first 拓扑序，但仍使用 best-fit，并在两种 victim 策略中选择，其中包括非对称策略，因此它不是完整的代价无感 planner。选定拓扑序在 3,072 为 capfit-id，从 3,584 起为 unlock-frontier。流量接近零时比值不稳定，故表 S13 保留原始值并将零分母留空。该扫描只覆盖 Conv_Case0、L1、固定 $H = 0$ 与流量指标；现有记录不含逐点 C/D 、溢出次数、时间或 canonical 验证字段，因而不外推为跨算子、跨缓存的鲁棒性结论。

4.7 自有合成基准的标准验证复评

早期固定 $H = 0$ 的内部 assign/extra-split 诊断没有 canonical 验证，现已由本节的标准复评取代，不再作为论文结果。我们在不改动任何实例的前提下，以完整生产求解器重跑全部三个自有合成集：36 例套件（目标缓存覆盖 UB、L0C 与 L1，容量受限与序可规避两种体制各 18 例）、8 例小规模 oracle 集（各 17 节点），以及 clean/dirty 镜像对。36 例的每个实例评估 7 个配置，共 252 次运行；脚本在每次运行生成调度、地址与溢出表后立即调用 canonical evaluator，CSV 记录的 252 行均为 valid=True、零违规。指标文件为 results/paper/v2_synth_suite.csv 等四个表，由 scripts/paper/v2_synth_suite.py 再生。

表 S14: 生产求解器在 36 例合成套件上的标准验证比较。前代组合指 capfit-id/P1/id-raw 三个候选序加距离驱逐策略的完整组合选择；四个代价无感序基线沿用原合成套件的定义；random-best 取 8 个种子随机拓扑中的最优。中位比值为本文 Tr 除以比较者 Tr (0/0 计为 1)。

比较者	全部 36 胜/平/负	容量受限 18	序可规避 18	中位比值
前代组合	0/36/0	0/18/0	0/18/0	1.00
cp-list	30/6/0	18/0/0	12/6/0	0.39
cp-free-first	14/22/0	14/4/0	0/18/0	1.00
pressure-uniform	13/23/0	10/8/0	3/15/0	1.00
goodman-hsu	19/17/0	7/11/0	12/6/0	0.67
random-best	27/9/0	18/0/0	9/9/0	0.34

三个结论。其一，生产求解器与前代组合在 36 例上完全持平。由于前代配置是生产组合候选集的子集，这一零负关系由组合单调性保证；标准验证确认了实现没有破坏该包含关系，但不能把它当作独立泛化证据。新模块在该合成族上未表现出相对前代的额外收益。其二，逐实例核对还表明 cp-free-first 序与生产组合中的 id-raw 序在 36/36 例上相同，且其距离策略、 $H = 0$ 也属于生产搜索空间；因此相对它的 14 胜 22 平同样是候选包含下的非回归检查。真正未被生产候选直接包含的是 cp-list、pressure-uniform、goodman-hsu 与 best-of-8 随机序；生产求解器相对这四组比较均为零负，但该观察只适用于这里固定的实现和随机预算。其三，记录在案的全部 252 行（36 例 $\times$ 7 个求解器配置）均通过 canonical 验证、零违规，该合成族的证据等级由“内部诊断”升级为“标准验证”。

表 S15–S16 给出全部 36 例的逐实例记录。C/R 表示容量受限/序可规避； C, D 表示有备份/生成缓冲的溢出体积；“前代”为完整前代组合，random 取 best-of-8。

表 S15: 36 例逐实例结果（第一部分）：实例属性、生产结果及前代组合。所有数值均为字节；次数为生产结果的溢出条目数。

实例	体制	缓存	节点	本文 Tr	C	D	次数	前代 Tr
synth_000	C	L0C	35	224	224	0	2	224
synth_001	R	L1	38	0	0	0	0	0
synth_002	C	UB	40	1,088	0	544	2	1,088
synth_003	R	L0C	40	0	0	0	0	0
synth_004	C	L1	34	1,376	1,376	0	2	1,376
synth_005	R	UB	30	0	0	0	0	0
synth_006	C	L0C	39	448	0	224	2	448
synth_007	R	L1	32	0	0	0	0	0
synth_008	C	UB	44	544	544	0	2	544
synth_009	R	L0C	46	0	0	0	0	0
synth_010	C	L1	38	2,752	0	1,376	2	2,752
synth_011	R	UB	24	0	0	0	0	0

续下页

表 S15 (续)

实例	体制	缓存	节点	本文 Tr	C	D	次数	前代 Tr
synth_012	C	L0C	43	224	224	0	2	224
synth_013	R	L1	38	0	0	0	0	0
synth_014	C	UB	28	1,088	0	544	2	1,088
synth_015	R	L0C	40	0	0	0	0	0
synth_016	C	L1	42	1,376	1,376	0	2	1,376
synth_017	R	UB	30	0	0	0	0	0
synth_018	C	L0C	47	448	0	224	2	448
synth_019	R	L1	32	0	0	0	0	0
synth_020	C	UB	32	544	544	0	2	544
synth_021	R	L0C	46	0	0	0	0	0
synth_022	C	L1	46	2,752	0	1,376	2	2,752
synth_023	R	UB	24	0	0	0	0	0
synth_024	C	L0C	31	224	224	0	2	224
synth_025	R	L1	38	0	0	0	0	0
synth_026	C	UB	36	1,088	0	544	2	1,088
synth_027	R	L0C	40	0	0	0	0	0
synth_028	C	L1	50	1,376	1,376	0	2	1,376
synth_029	R	UB	30	0	0	0	0	0
synth_030	C	L0C	35	448	0	224	2	448
synth_031	R	L1	32	0	0	0	0	0
synth_032	C	UB	40	544	544	0	2	544
synth_033	R	L0C	46	0	0	0	0	0
synth_034	C	L1	34	2,752	0	1,376	2	2,752
synth_035	R	UB	24	0	0	0	0	0

表 S16: 36 例逐实例结果 (第二部分): 五种比较顺序的 Tr。random 为 best-of-8。

实例	cp-list	cp-free-first	pressure	Goodman-Hsu	random
synth_000	560	336	224	224	688
synth_001	1,024	0	0	256	0
synth_002	2,464	2,176	1,520	1,632	3,984
synth_003	384	0	192	48	336
synth_004	2,064	1,376	1,376	2,064	2,064
synth_005	0	0	0	0	0
synth_006	1,184	896	800	448	1,152
synth_007	1,376	0	0	688	688
synth_008	1,936	1,360	1,104	1,088	2,896
synth_009	256	0	0	32	160
synth_010	5,504	2,752	2,752	2,752	5,856
synth_011	0	0	0	0	0
synth_012	688	448	336	336	1,024
synth_013	1,024	0	0	256	0
synth_014	1,632	1,632	1,088	1,088	1,664
synth_015	384	0	192	48	336
synth_016	3,440	2,064	1,376	1,376	4,480

续下页

表 S16 (续)

实例	cp-list	cp-free-first	pressure	Goodman-Hsu	random
synth_017	0	0	0	0	0
synth_018	1,312	896	864	672	1,696
synth_019	1,376	0	0	688	688
synth_020	1,088	1,088	816	544	1,504
synth_021	256	0	0	32	160
synth_022	8,256	5,504	3,104	2,752	7,936
synth_023	0	0	0	0	0
synth_024	448	224	336	224	448
synth_025	1,024	0	0	256	0
synth_026	2,320	2,176	1,376	1,088	3,440
synth_027	384	0	192	48	336
synth_028	4,480	2,752	1,376	1,376	5,872
synth_029	0	0	0	0	0
synth_030	1,120	672	448	448	1,152
synth_031	1,376	0	0	688	688
synth_032	1,376	1,088	960	816	2,048
synth_033	256	0	0	32	160
synth_034	4,128	2,752	2,752	4,128	4,128
synth_035	0	0	0	0	0

表 S17: 8 例 oracle 集上生产求解器与其自身所选拓扑序的固定序 CP-SAT 流量最优值。全部 8 例求解器输出恰好等于最优值（差距为零，状态均为 OPTIMAL）。

实例	受限缓存	本文 Tr	固定序最优	差距
oracle_000	L0C	352	352	0
oracle_001	UB	1,408	1,408	0
oracle_002	L0C	704	704	0
oracle_003	UB	704	704	0
oracle_004	L0C	352	352	0
oracle_005	UB	1,408	1,408	0
oracle_006	L0C	704	704	0
oracle_007	UB	704	704	0

oracle 集给出一个公开实例上无法整体获得的量化标尺：对每个 17 节点实例，以生产求解器实际选出的拓扑序运行第 2 节的固定序 CP-SAT 模型，8 例全部求得最优并与求解器输出严格相等（表 S17）。即在该小规模族上，可扩展驱逐启发式没有在其自身拓扑序下留下任何流量余量。该结论仅覆盖 17 节点实例与流量目标，不外推到公开规模——Conv_Case0 上生产结果 66,828 相对同序证书 57,408 的 14.1% 差距（正文第 5 节）表明大规模实例上余量确实存在。

clean/dirty 镜像对进一步以最小算例复核 $Tr = Vol + Dt$ 的计费语义：两个实例的溢出体积同为 $Vol = 1,024$ 、溢出次数同为 2，clean 版全部有备份（ $Dt = 0$ ， $Tr = 1,024$ ），dirty 版全部为生成缓冲（ $Dt = 1,024$ ， $Tr = 2,048$ ），计费严格加倍；生产求解器与前代组合在两例上持平。